

SemanticCache Poisoning

SemanticCache Poisoning - dequeueing, GitHub.

- Published: date not stated
- Original: <https://github.com/dequeueing/SemanticCache_Poisoning>
- Preserved from: https://github.com/dequeueing/SemanticCache_Poisoning (git) on 2026-08-19
- Repository commit: 1fd553d34a5d4fe115a26d8aa45c5aaeac8cfece
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <https://github.com/dequeueing/SemanticCache_Poisoning>
- Commit: 1fd553d34a5d4fe115a26d8aa45c5aaeac8cfece
- Documents preserved: 1

README.md

Blob e2d06fb3e0a3 , 3038 bytes, at commit 1fd553d34a5d .

Semantic Cache Poisoning

Official repository of NDSS 2026 paper: [When Cache Poisoning Meets LLM Systems: Semantic Cache Poisoning and Its Countermeasures](#).

The first in-depth analysis of semantic cache poisoning in LLM systems.

Attack Overview

[image: Attack Overview]

Quick Usage

Setup Environment

This project uses `uv` for dependency management. Make sure you have **Python 3.12+** and `uv` installed.

1. Install `uv` (if not already installed):

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

2. Create a virtual environment and install the project:

```
uv venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
uv pip install -e .
```

This installs all dependencies declared in `pyproject.toml` and makes the `src` package importable from anywhere in the project.

Your Own Dataset (Optional)

We have prepared the selected dataset under `data/`. Change the content if you want to run your own dataset.

```
uv run python download_dataset.py
```

Set API Key

Please checkout `src/config.py`, `src/llm.py` and `src/embedding.py` to see the API, LLM and embedding model setting. You can change the backend if needed.

Run Attacks

Use `main.py` to launch the attack:

```
uv run python main.py
```

Configuration

All tuneable hyperparameters are centralised in `src/config.py`, you can change them if needed.

Parameter	Default	Description
<code>LLM_MODEL</code>	<code>qwen3.5-plus</code>	LLM model name
<code>LLM_TEMPERATURE</code>	<code>1.0</code>	Sampling temperature
<code>LLM_MAX_TOKENS</code>	<code>200</code>	Max tokens per LLM response
<code>EMBEDDING_MODEL_NAME</code>	<code>distilbert-base-uncased</code>	Embedding model (HuggingFace ID)

Parameter	Default	Description
GCG_NUM_ITER	120	Number of GCG optimisation iterations
GCG_THRESHOLD	0.8	Cosine similarity threshold for attack success
GCG_BATCH_SIZE	512	Candidate batch size per GCG step
GCG_TOPK	256	Top-k token candidates per position
GCG_DEVICE	cuda	Torch device for white-box attack

A Side Note

For ease of reproduction, this codebase does not deploy a real semantic cache. Instead, it simulates the semantic cache workflow by assuming an empty cache at the start of each run. As demonstrated in Section VI.B of our paper, this simplification has minimal impact on attack effectiveness. Note, however, that the evaluation statistics reported in the paper are collected under real-world semantic cache deployments.

Acknowledgement

- Our whitebox implementation references [Universal and Transferable Attacks on Aligned Language Models](#).

Citation

If you use this code, please cite our [paper](#)

Archived from https://github.com/dequeueing/SemanticCache_Poisoning. Rendered offline from the local Markdown copy.